

The Redemption — SRE Design Document

Service: business-critical point-redemption microservice on AWS EKS. **Targets:** zero downtime; absorb sudden **10x flash-sale spikes** with no manual intervention; protect customer data.

Architecture diagram: docs/architecture.drawio (open in draw.io → export PNG). IaC: this repo (Terraform + Kustomize/Argo CD). App + CI: redemption-app repo.

Executive summary

A single Express/PostgreSQL service is deployed to **EKS across 3 AZs**, fronted by an **ALB (WAF + ACM TLS)** and backed by **Aurora PostgreSQL Serverless v2 through an RDS Proxy**. Elasticity is two-layered: **HPA** scales pods on CPU *and* a real request-rate metric, while **Karpenter** scales nodes (spot + on-demand, Graviton + x86). Everything is delivered by **GitOps (Argo CD)** from immutable, git-SHA-tagged images. Security follows least-privilege (IRSA, scoped SGs, restricted PSS) and defense-in-depth (3-tier VPC, default-deny NetworkPolicy, WAF, KMS everywhere). Observability is Prometheus/Grafana/Alertmanager with SLO alerts routed to Slack/PagerDuty.

The stack runs as **two fully isolated environments from one codebase**: **dev** (continuous deployment from the `develop` branch) and **prod** in a **separate AWS account** (deploys gated on `v*` release tags), sharing only the git repos and DNS zone — per-env Terraform inputs and Kustomize overlays carry every difference.

A. Compute & Architecture

- EKS** managed control plane (KMS-encrypted etcd, audit + authenticator logs). Workers in **private subnets** only.
- Two node tiers**: a small always-on **managed node group** (Graviton `m6g`) provides baseline capacity for cluster-critical add-ons; **Karpenter** adds elastic burst capacity for the 10x spike. At low load the app shares the baseline nodes; a taint/affinity split can dedicate Karpenter nodes to the app where stricter isolation is required.
- App resilience**: Deployment with `maxUnavailable: 0` / `maxSurge: 1` (zero-downtime rolling), **PodDisruptionBudget**, **topology spread** across AZs (`DoNotSchedule`) and nodes, and startup/readiness/liveness probes with graceful SIGTERM shutdown. Hook Jobs (migrations) carry their **own label** so they never poison the PDB/Service selectors (a Job-owned pod in a PDB selector zeroes its allowed disruptions).
- Data: Aurora Serverless v2** (writer + reader, Multi-AZ) fronted by **RDS Proxy** so a large pod fan-out does not exhaust DB connections.
- Images**: multi-arch (arm64/amd64) so pods run on the cheaper Graviton nodes *and* x86 spot.

B. Scalability strategy (the 10x spike)

Layer	Mechanism	Behaviour
Pods	HPA v2 on CPU (60%) + <code>http_requests_per_second</code> (custom metric via prometheus-adapter)	Aggressive scale-up (double or +10 pods / 15s, 0s stabilization); slow scale-down (5-min window) to avoid flapping when a sale ends
Nodes	Karpenter	Provisions spot+on-demand across arm64/amd64 in seconds; consolidates after the spike
Database	Aurora Serverless v2 (0.5→32 ACU dev, up to 64 in prod) + RDS Proxy	DB capacity follows load; proxy pools/multiplexes connections

The RPS metric reacts to the real traffic signal faster than CPU alone — critical for a spiky, latency-sensitive redemption path.

C. Security & networking (least privilege + defense in depth)

- Network tiers**: public (ALB/NAT only), private (workers, egress via NAT), **isolated DB subnets** (no internet route). Security groups are chained: app → RDS Proxy → Aurora, each accepting only the previous hop.

- **Pod layer: default-deny NetworkPolicy** with explicit allows (ALB→3000, Prometheus scrape, DNS, DB egress scoped to the VPC CIDR); **Pod Security Standard restricted** ; non-root, read-only rootfs, all capabilities dropped, seccomp `RuntimeDefault` .
- **Identity: IRSA** with per-workload least-privilege (e.g. External Secrets can read exactly one secret + decrypt one KMS key). CI authenticates to AWS via **GitHub OIDC** — no static keys.
- **Edge: WAF** web ACL + **ACM TLS** on every ALB. The public app has its own ALB; **Grafana and Argo CD share one CIDR-allowlisted admin ALB** (ALB IngressGroup) — admin planes are never exposed on the public app path. EKS API endpoint **lockable to admin CIDRs** via variable (private access always on).
- **Data protection: KMS** encryption for etcd secrets, Aurora, ECR, and EBS volumes; DB credentials in Secrets Manager, synced read-only into the cluster by External Secrets.
- **Supply chain: ECR immutable** tags + scan-on-push; CI **Trivy** scan gates the push; deploys pin immutable git-SHA image tags.

D. Reliability & observability

Failure recovery:

Scenario	Response
AZ outage	Pods are spread across 3 AZs (topology spread), so a lost AZ removes only ~1/3 of capacity; the HPA + Karpenter reprovision the shortfall in healthy AZs; Aurora fails over to a standby AZ.
Bad deployment	Rolling update keeps full capacity; readiness gating stops a broken image from serving; rollback = <code>git revert</code> (Argo CD re-syncs) — immutable tags make it deterministic. Migrations are decoupled from the app lifecycle : schema changes run as a one-off Kubernetes Job (Argo CD PreSync hook) <i>before</i> the new pods deploy — never on app boot — so an HPA burst of dozens of pods during a 10× spike can't stampede the DB with concurrent migrations (avoiding lock contention and connection exhaustion). Migrations stay strictly backward-compatible (expand → contract) so the running old version and the newly deploying version share the one Aurora seamlessly through the rolling update. <i>(Optional: Argo Rollouts canary with metric analysis.)</i>
Node loss / drain / spot reclaim	Karpenter (interruption queue) drains and replaces nodes; the PodDisruptionBudget (<code>maxUnavailable: 1</code>) + surge keep the service available during voluntary disruptions/consolidation.
DB pressure	RDS Proxy shields Aurora from connection storms; Serverless v2 scales ACUs.

Observability (SLIs): availability, **5xx error rate**, **p99 latency**, **RPS**, saturation (CPU/mem). Collected by kube-prometheus-stack; the app exposes `/metrics` . Alerting: 11 PrometheusRules (platform: no-replicas, crashloop, OOM, HPA-maxed, pending; app: high error-rate, high p99) → Alertmanager → **Slack (all) / PagerDuty (critical)**. Control-plane logs + VPC flow logs to CloudWatch.

SLOs (the targets the alerts defend): availability $\geq 99.9\%$, p99 latency **< 300 ms**, 5xx error rate **< 0.1%** — critical alerts page when the error budget is at risk; warnings notify Slack.

E. Operations

Day-2 (minimise toil):

- **GitOps** (Argo CD `selfHeal`) — cluster state = git; drift auto-corrects; PR-based changes with review + audit trail.
- **Managed everything** — EKS control plane, managed add-ons, Aurora Serverless, Karpenter consolidation (cost + capacity self-tuning), ECR lifecycle expiry.
- **Automated CD** — CI builds/scans/pushes and bumps the image tag; no manual `kubectl apply` .
- **IaC** — one Terraform root module + per-env inputs (S3 remote state, DynamoDB locking); dev and prod are stamped from the same code, so environment drift can't accumulate.
- **Cost efficiency** — Spot + Graviton nodes, Aurora Serverless v2 scaling to a 0.5-ACU floor, Karpenter consolidation, and a shared admin ALB (one ALB for Grafana + Argo CD) keep spend proportional to load — important when baseline traffic is low but flash sales spike 10×.

Team delegation (1 Senior + 2 Juniors):

Engineer	Ownership
Senior (lead/reviewer)	Foundation & guardrails: VPC/EKS/IAM/KMS Terraform, Karpenter, RDS Proxy + Aurora, GitOps bootstrap, PR review, security posture, on-call runbooks.

Engineer	Ownership
Junior A (app platform)	App Deployment/Service/Ingress/HPA/PDB, Kustomize overlays, the <code>redemption-app</code> Dockerfile + CI pipeline, <code>/metrics</code> instrumentation.
Junior B (observability & networking)	ServiceMonitor/PrometheusRule/Alertmanager, Grafana dashboards, NetworkPolicies, ExternalSecrets wiring, DNS/cert automation.

Juniors work in parallel behind clear interface contracts (image tag, service name/port, metric names); the Senior owns anything cluster-wide or security-sensitive and reviews every PR.

Key trade-offs

Decision	Chosen	Trade-off / rejected alternative
DB	Aurora Serverless v2 + RDS Proxy	Auto-scaling + connection safety vs. slightly higher cost than a fixed instance.
Rollout	Rolling (zero-downtime)	Simple & capacity-preserving; canary via Argo Rollouts deferred (adds a controller) but is the natural next step for stronger bad-deploy protection.
Nodes	Karpenter, spot-first, multi-arch	Cost + fast bursts vs. spot interruption risk (mitigated by on-demand fallback + PDB).
GitOps source	Separate <code>redemption-infra</code> repo	Clean app/infra separation vs. an extra cross-repo CI write (needs a scoped token).
Environments	Prod in a separate AWS account	Hard blast-radius/IAM isolation and independent quotas vs. duplicated infra cost and a second bootstrap. Prod images are rebuilt from the release tag and pushed to prod's own ECR (self-contained, merge-strategy-proof) rather than digest-promoted from dev.
Compute	EKS	Portability + ecosystem vs. more moving parts than ECS/Fargate.

Deploy (summary)

- Per environment: `terraform init -backend-config=envs/<env>/backend.hcl && terraform apply -var-file=envs/<env>/terraform.tfvars` (VPC, EKS, Aurora, Karpenter, add-ons).
- Bootstrap the cluster's own Argo CD + apply its application set (`argocd/dev/` or `argocd/prod/`) → cluster converges from git.
- Dev:** push `develop` → CI builds multi-arch image, scans, bumps the dev overlay → Argo CD deploys. **Prod:** tag `v*` on `main` → CI rebuilds/scans the tagged commit, pushes to the prod account's ECR, bumps the prod overlay → prod Argo CD deploys.

Full runbooks: `SETUP.md` (dev) and `PROD-SETUP.md` (prod account).